



TM233

B&R Application Design Guidelines

Table of contents

1 Motivation.....	4
2 Objectives.....	5
3 Reading rules.....	6
4 Rules A001-A005: Project structure.....	7
4.1 Rule A001: Place source code in "Source" package (Logical View).....	7
4.2 Rule A002: Place documents in "Documentation" package (Logical View).....	8
4.3 Rule A003: Place simulations in "Simulation" package (Logical View).....	9
4.4 Rule A004: Place user content inside "UserFiles" package (Logical View).....	10
4.5 Rule A005: Place .typ file before usage in .var file (Logical View).....	11
5 Rules A101-A104: PLC performance.....	12
5.1 Rule A101: Choose and configure task classes.....	12
5.2 Rule A102: Synchronize task class timing with IO bus.....	13
5.3 Rule A103: Use most recent available versions.....	14
5.4 Rule A104: Disable module supervision but monitor "ModuleOk"	15
6 Rules A201-A203: mapp Technology.....	16
6.1 Rule A201: Use prefixes for naming MpLink variables.....	16
6.2 Rule A202: Insert mapp View link into Logical View.....	18
6.3 Rule A203: mapp configuration file naming.....	19
7 Rules A301-A306 Coding principles.....	20
7.1 Rule A301: Avoid hard-coded literals.....	20
7.2 Rule A302: Naming of .var and .typ files.....	21
7.3 Rule A303: Release dynamic resources in the _EXIT.....	23
7.4 Rule A304: Don't disable mapp Motion function blocks in the _EXIT.....	24
7.5 Rule A305: Avoid bottom-up dependencies.....	25
7.6 Rule A306: Set parameters before giving command.....	26
8 Rules A401-A408: Modularity.....	27
8.1 Rule A401: Well-defined interfaces.....	27
8.2 Rule A402: Enable input for each software module.....	30
8.3 Rule A403: Create a local copy of the interface structure.....	31
8.4 Rule A404: IO-module names in Physical View should correspond to design documents.....	32
8.5 Rule A405: Mirror Logical View folder structure in Configuration View.....	33
8.6 Rule A406: Unique *.sw, *.iom, *.vvm file per module.....	34
8.7 Rule A407: Unique OpcUaMap (.uad) file for every module.....	35
8.8 Rule A408: Unique *.binding and *.eventbinding file for per *.content file (mapp View).....	36
9 Rules A501-A502: Documentation.....	37
9.1 Rule A501: Include a "Readme" file.....	37
9.2 Rule A502: List abbreviations used in the project.....	39
10 Rules A601-A601: Testing.....	40
10.1 Rule A601: Use the IEC Check library during development.....	40
11 Rules A701-A702: Usability.....	42

11.1 Rule A701: Make SDM available on the HMI.....42

11.2 Rule A702: Global variables availability in Watch window.....43

1 Motivation

The Power of Software Guidelines in Machine Automation Projects

Modularity

Modularity refers to breaking down complex systems into smaller, independent components. In the context of machine automation software, modularity offers several advantages:

- **Reusability:** Modular components can be reused across different projects, saving development time and effort. For instance, a well-defined motor control module can be reused in various robotic applications.
- **Maintenance Ease:** When a specific module needs updates or fixes, you can focus on that isolated part without affecting the entire system. This simplifies maintenance and reduces the risk of unintended side effects.
- **Testing Isolation:** Modular components can be tested independently, ensuring that each piece functions correctly before integration. This promotes robustness and reliability.

Teamwork

Collaboration among engineers is crucial for successful automation projects. Software guidelines facilitate effective teamwork:

- **Consistency:** Guidelines ensure that all team members follow a common approach. Consistent coding styles, documentation practices, and naming conventions lead to better communication and understanding.
- **Reduced Friction:** When everyone adheres to the same guidelines, code reviews become smoother, and merging contributions is less error-prone.
- **Knowledge Sharing:** Guidelines encourage knowledge sharing. New team members can quickly grasp existing codebases, leading to faster onboarding.

Scalability

Scalability is essential as automation systems grow in complexity:

- **Flexible Architecture:** Guidelines help design software that can scale seamlessly. Whether you're adding more sensors, actuators, or modules, a well-structured architecture accommodates growth.
- **Efficient Resource Utilization:** Scalable software optimizes resource usage. It adapts to varying workloads, ensuring efficient utilization of computational resources.
- **Future-Proofing:** Scalable systems are ready for future enhancements. As technology evolves, your software can evolve too, without major overhauls.

Software guidelines empower automation engineers to build robust, maintainable, and adaptable solutions. By embracing modularity, fostering teamwork, and prioritizing scalability, we pave the way for a smarter, more efficient future in machine automation.

Remember, these guidelines are not rigid rules but rather flexible tools that empower engineers to create exceptional software. Let's continue to innovate, collaborate, and drive progress in the exciting field of automation!

2 Objectives

The application design guidelines will help you optimize the quality of your application projects. You will be able to:

- Recognize why application guidelines are crucial for creating maintainable, scalable, and robust software.
- Explain the concept of modular design and its benefits.
- Identify techniques for breaking down complex systems into manageable modules.
- Practice strategies for writing code that is easy to maintain and extend.
- Demonstrate best practices for handling errors in software design.
- Design a robust application.
- Recognize the importance of clear documentation.
- Create applications that facilitate collaboration with cross-functional teams.
- Analyze existing projects to identify design strengths and weaknesses.
- Evaluate and refactor existing code based on design guidelines.

To achieve high software quality for your projects, refer also to:

- [TM231 – B&R Coding Guidelines](#)
- [TM232 – B&R Library Design Guidelines](#)

3 Reading rules

Overview of rules

These guidelines are divided into the following categories:

- Project structure
- PLC performance
- mapp Technology
- Coding principles
- Modularity
- Documentation
- Testing
- Usability

How to read the rules

Each rule can be identified by a “Rule ID” which is a unique 3-digit number.

Rule ID (3-digit): Name of rule

Guidelines

Suggested guidelines are stated here.

Examples

This section is for relevant code examples.

NO:

```
code example here
```

YES:

```
code example here
```

Reasoning

Why is this guideline necessary? What are the benefits? This section contains answers to these questions.

Exceptions and comments

If there are any exceptions to the stated guideline or if there are any additional comments, you can find them in this section.

References

References from literature are mentioned here.

4 Rules A001-A005: Project structure

This section provides you with guidelines for the design and organization of software modules within Automation Studio.

A well-organized project structure makes it easier to maintain and enhance the software over time. As the project grows, a clear structure allows for seamless additions of new features or modules. Configuration files, packages, user-defined structures, etc. should be organized systematically, making it easier to understand the software architecture. This includes dividing the project into meaningful packages, grouping related files together, and using consistent naming conventions to understand the purpose of each component.

4.1 Rule A001: Place source code in "Source" package (Logical View)

Guidelines

All source code files and packages should be placed in a separate package called "Source" in Logical View.



This guideline is meant as a suggestion/proposal and not as a mandatory rule.

Examples

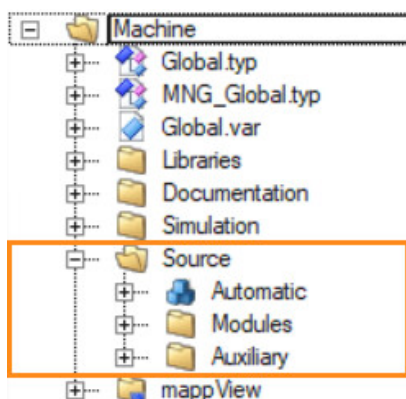


Figure 1: "Source" package in Logical View

Reasoning

Placing all the source files in one package makes the productive code distinguishable from other packages like Documentation, Simulation, etc.

Exceptions and comments

None

References

None

4.2 Rule A002: Place documents in "Documentation" package (Logical View)

Guidelines

Documents relevant for a project should be part of the Automation Studio project and placed into a dedicated package in the Logical View. This package should be named appropriately, the suggested name is "Documentation". The main focus lies on the developer documentation, however all the following documents can be part of this folder:

- Software design overview
- List of abbreviations
- Project.language file
- Change log
- HMI user manual
- List of alarms
- List of custom errors (Range of the "Error IDs", Type of error → "info", "warning" or "error", Severity, Description)



This guideline is meant as a suggestion/proposal and not as a mandatory rule.

Examples

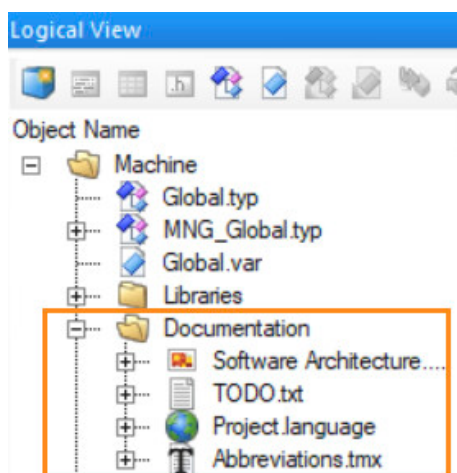


Figure 2: "Documentation" package in Logical View

Reasoning

The AS project should be a "one-stop-shop" for any new developer working on a project. Everything a new developer needs to understand the project, they will find inside the AS project.

Exceptions and comments

Large binary files (videos, etc.) should not be added to the AS project.

If the project is under source control and usually handed over as a complete repository, you might want to place the Documentation folder outside of the project (on the same level as the AS project itself).

If some of the documentation need to be accessed and edited by other people than the AS developer team (e.g. the Machine Operator's Manual), it might be a better idea to place also the documentation outside of the AS project.

References

None

4.3 Rule A003: Place simulations in "Simulation" package (Logical View)

Guidelines

Simulation models from external tools (iPhysics, Robot Studio, Scene Viewer ...) should be stored in a separate package named "Simulation" in the Logical View.

Examples

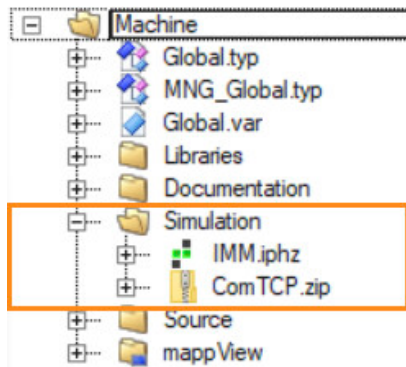


Figure 3: "Simulation" package in Logical View

Reasoning

The AS project should be a "one-stop-shop" for any new developer working on a project. Everything a new developer needs to understand the project, they will find inside the AS project.

Exceptions and comments

If the simulations are composed of large binary files or have complex folder structures, they should not be added to the AS project.

If the project is under source control and usually handed over as a complete repository, you might want to place the Simulation folder outside of the project (on the same level as the AS project itself).

References

None

4.4 Rule A004: Place user content inside "UserFiles" package (Logical View)

Guidelines

All contents which should be stored e.g. on the user partition of the Compact Flash (typically: recipe files, configuration files, ...) and are needed for a correct start-up of the PLC should be stored well visible in the Logical View in a "UserFiles" package.



This guideline is meant as a suggestion/proposal and not as a mandatory rule.

Examples

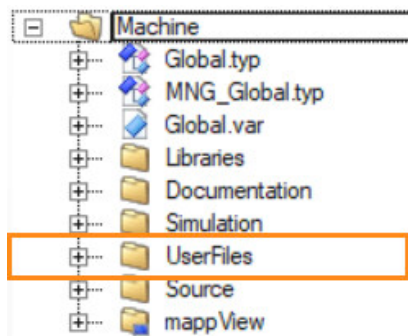


Figure 4: "UserFiles" package in Logical View

Reasoning

These files may be necessary for a correct start-up of the application (e.g. configuration data for the machine, or a valid default recipe) and must therefore be stored on the PLC/CompactFlash during the installation procedure.

Exceptions and comments

None

References

None

4.5 Rule A005: Place .typ file before usage in .var file (Logical View)

Guidelines

The .typ file must be placed in the Logical View before the .var file.
This guideline applies to global, package-global and local scope.

Examples



Figure 5: File structure of a program in the Logical View with .typ files placed above ".var" file

Reasoning

With GNU compiler 6.3 the size of a variable of type "enum" in a C or C++ task is defined according to the number of enum elements and the assigned values. That's why the GNU compiler 6.3 needs the .typ file to be placed before the .var file, so that the enum type information is available for a correct variable declaration.

Exceptions and comments

None

References

None

5 Rules A101-A104: PLC performance

Optimizing PLC performance involves a combination of software design, efficient coding practices, and adherence to industry standards. By following the guidelines in this section, you can ensure your PLC operates efficiently with a high performance level.

5.1 Rule A101: Choose and configure task classes

Guidelines

Choose and configure the right task classes for your programs to ensure optimal CPU performance at an early stage of the project.

The guideline includes best practices for assigning programs to the correct task classes and configuring task classes (duration, tolerance).

Examples

Best practices

- The most critical machine equipment that influences quality and machine performance should run in the highest task class.
- Other mechatronic units of the machine should run in fast task classes, too.
- The automatic sequence control (typically a big state machine) should also run in a fast task class, because otherwise unnecessary time is lost during production cycle at every switch to a new Case in the state machine.
- Mechatronic units that do not need high speed calculations can run in slower task classes. A tight tolerance is set to ensure realtime behavior.
- Infrastructure/service programs should also run in a reasonable cycle time to ensure fast reaction on user input (e.g. login button, recipe load button), however, they do not need realtime. In tight situations they could also run slower without influencing the quality of the machine output, so a very high tolerance is set.

Object Name	Version	Transfer To	Size (...	D...	Source	Source File	Description
<CPU>							
Cyclic #1 - [0.4 ms]							
Mold	1.00.0	UserROM	0		Source.Modules.Mold.Mold	CP1686X\X20CP1686X\Cpu.sw	Mold control program
Injection	1.00.0	UserROM	0		Source.Modules.Injection.Injection	CP1686X\X20CP1686X\Cpu.sw	Injection control program
Cyclic #2 - [2 ms]							
Automatic	1.00.0	UserROM	0		Source.Automatic	CP1686X\X20CP1686X\Cpu.sw	Automatic sequence (machine production cycle)
Ejector	1.00.0	UserROM	0		Source.Modules.Ejector.Ejector	CP1686X\X20CP1686X\Cpu.sw	Ejector control program
Cyclic #3 - [50 ms]							
Heating	1.00.0	UserROM	0		Source.Modules.Heating.Heating	CP1686X\X20CP1686X\Cpu.sw	Heating control program
Cyclic #4 - [100 ms]							
Alarm	1.00.0	UserROM	0		Source.Auxiliary.Alarm.Alarm	CP1686X\X20CP1686X\Cpu.sw	Alarm management program
Cyclic #5 - [200 ms]							
Cyclic #6 - [500 ms]							
Cyclic #7 - [1000 ms]							
Cyclic #8 - [124 ms]							
Recipe	1.00.0	UserROM	0		Source.Auxiliary.Recipe.Recipe	CP1686X\X20CP1686X\Cpu.sw	Recipe management program

Figure 6: Distribution of tasks to different task classes

Reasoning

The assignment of programs to task classes has a big impact on the performance of the machine.

The assignment of programs to task classes has a great influence on the behavior of the application, since tasks of the same task class are executed one after the other, while tasks of different task classes can interrupt each other.

Exceptions and comments

The Profiler is a powerful tool and can be used to measure important system data such as task runtimes, system loads, and so on. This can be an indicator of how much idle time remains in a task class and whether there is an overload that can be avoided.

References



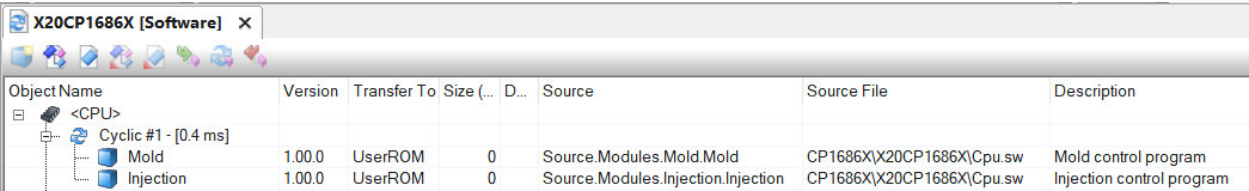
B&R Online Help Link: [Runtime performance](#)

5.2 Rule A102: Synchronize task class timing with IO bus

Guidelines

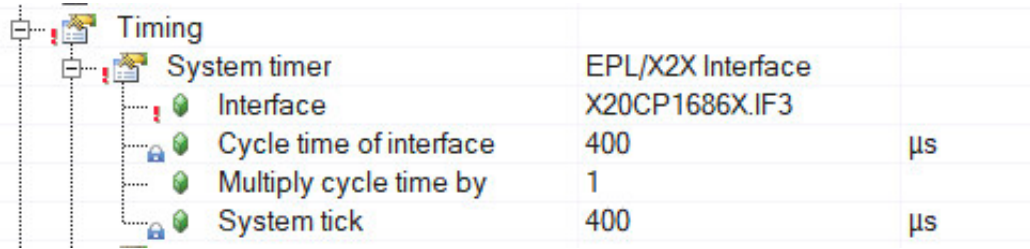
The Automation Runtime task class system should be synchronized with the IO bus. The same hardware clock should be used for the entire runtime system.
Multiples of the IO bus time can be used to slow down the task class system of the PLC, if necessary.

Examples



Object Name	Version	Transfer To	Size (...)	D...	Source	Source File	Description
<CPU>							
Cyclic #1 - [0.4 ms]							
Mold	1.00.0	UserROM	0		Source.Modules.Mold.Mold	CP1686X\X20CP1686X\Cpu.sw	Mold control program
Injection	1.00.0	UserROM	0		Source.Modules.Injection.Injection	CP1686X\X20CP1686X\Cpu.sw	Injection control program

Figure 7: Task class timings in the Logical View Figure.



Timing		
System timer	EPL/X2X Interface	
Interface	X20CP1686X.IF3	
Cycle time of interface	400	µs
Multiply cycle time by	1	
System tick	400	µs

Figure 8: Configuration of System Timer in the CPUConfiguration of System Timer in the CPU.

Reasoning

It is necessary to use the same hardware clock for the entire runtime system to ensure synchronized timing behavior between the cyclic system of Automation Runtime and the IO dispatching via the respective PLK or X2X networks.

Exceptions and comments

None

References

- [Real-time operating system / Target systems / Target systems - SG4 / I/O management / Method of operation/ I/O data synchronization / I/O Scheduler](#)
- [Real-time operating system / Target systems / Target systems - SG4 / I/O management / Method of operation/ I/O data synchronization / Synchronizing I/O bus systems](#)

5.3 Rule A103: Use most recent available versions

Guidelines

While developing a new project ensure to use of the latest upgrades.

Examples

None

Reasoning

Newer upgrades contain improved features and bug fixes from previous versions.

Exceptions and comments

It is important to consider the demands and constraints of the machine and consider customer requirement.

When upgrading a project under source control, make sure that your develop/main branch is locked and no other branches get merged into it, because version upgrades change binaries (libraries) and various lines in the *.hw file (hardware module upgrades).

References

None

5.4 Rule A104: Disable module supervision but monitor “ModuleOk”

Guidelines

The "Module supervised" configuration should be turned off. Connect a variable to the "ModuleOk"-Flag, to indicate the status of the IO module. This gives the application the possibility to react accordingly if an IO module is unplugged, e.g. by setting an alarm, or disabling a certain machine function.

Examples

Name	Value	Unit	Description
X20DO4322			
Function model	default		Module operating mode
General			
Module supervised	off		Service mode if there is no hardware module
Output status information	on		Additional output status information
Packed outputs	off		Packed I/O data instead of single digital outputs
Simulation			

Figure 9: "Module supervised" disabled in the I/O configuration.

Reasoning

In most application use cases it is an unwanted behavior that the PLC goes into service mode if an IO module is not plugged. To avoid unnecessary rebooting of the machine, the "Module supervised" configuration should be turned off.

Exceptions and comments

If a module is critical for operation of the machine, "Module supervised" should be "On".
Hint: The state of a powerlink slave can be detected by reading out the NMT_NodeAssignment_AU32 array (16#1F81) and NMT_MNNodeCurrState_AU8 array (16#1F8E) on the master (node:=0).
Module supervised for newly added modules can be disabled as a default option in PC properties - I/O

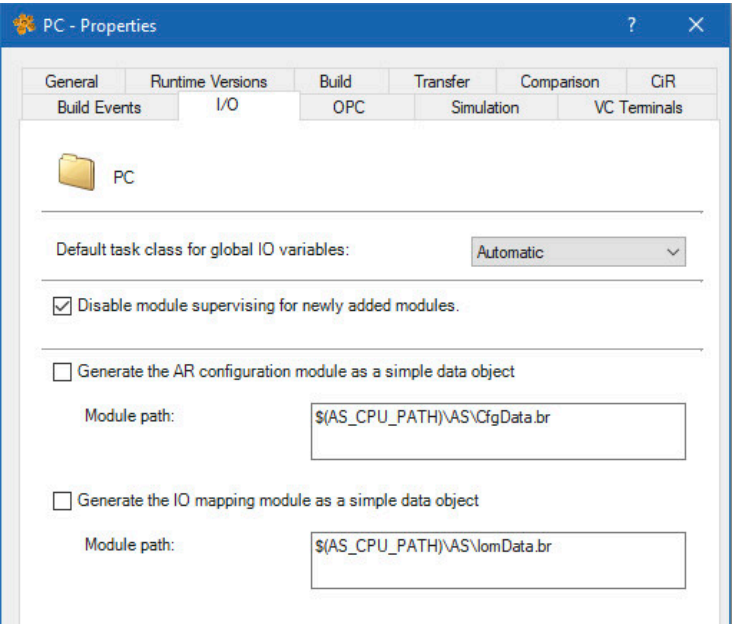


Figure 10: Disable module supervising in PC properties.

References

None

6 Rules A201-A203: mapp Technology

mapp Technology supports developers of machine and system applications by providing a way to efficiently implement recurring standard functions. The guidelines provided in this section will support you in using the mapp Technology components efficiently.

6.1 Rule A201: Use prefixes for naming MpLink variables

Guidelines

Use prefixes for naming MpLink variables.

Prefixes for MpLink-variables in the Configuration-View should follow one of the following guidelines (maintain consistency throughout the whole project):

- 1) "gMp"
- 2) "gMpLink"
- 3) Default name

Suffix suggestion:

In addition, if the MpLink needs to be described further, the purpose or function of the MpLink can be appended as a suffix, e.g. **gAlarmXCoreMain**, **gAlarmXCoreTank**, gAxisCutter, gAxisConveyor.

Examples

Suggestion	Example
"gMp"	gMpAlarmXCore
"gMpLink"	gMpLinkAlarmXCore
Default name	gAlarmXCore
Suffix	gAlarmXCoreMain

Reasoning

Automation engineers can be very "creative" when giving names to MpLink-variables:

A few examples of MpLink names used in different projects

- MpLinkPaintMixergAlarmCore
- gAlarmXCore
- MpLinkgAlarmXCore
- gRecipeManagement

For a professional presentation to our customers, we need to standardize the naming of MpLink variables.

- "gMp"

Benefits

- Auto completion (Ctrl-SPACE) provides all MpLinks while typing.
e.g. MpAlarmXSet(ADR(gMp ... Ctrl-SPACE shows all available MpLink-variables
- fits easier into the 32-character-limit

- "gMpLink"

Benefits

- Auto completion (Ctrl-SPACE) provides all MpLinks while typing.
e.g. MpAlarmXSet(ADR(gMpLi ... Ctrl-SPACE shows all available MpLink-variables

- Default name

Benefits

- simple and readable
- standard (easy to recognize for colleagues and customers)

Exceptions and comments

The prefix "g" is mandatory, because the MpLink is a global variable and global variables must carry the "g"-prefix see Rule C006 of the B&R Coding Guidelines.

References

Rule C006 of the B&R Coding Guidelines

6.2
Rule A202: Insert mapp View link into Logical View

Guidelines

Insert the mappView URL inside the Logical View.
The link should be placed as .url file as the first item in the "mapp View" package. The link file should be named "mappViewLink.url".



This guideline is meant as a suggestion/proposal and not as a mandatory rule.

Examples

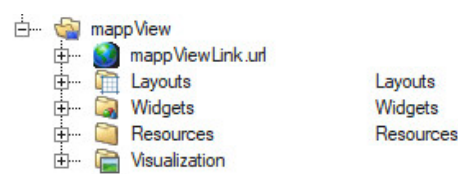


Figure 11: mapp View link as URL.

Reasoning

For a project engineer, working on the project the access to the mapp View HMI is faster.
For externals (reviewers, new team members), accessing the mapp View HMI is much faster as they save the time of looking up the "Visualization id" in the Configuration View.

Exceptions and comments



In Windows, you can create a .url file that points to a web address by dragging the address from your web browser to your desktop.

Another option is to configure the HMI application as the default HMI application so that it can be addressed by the browser without specifying the visualization ID in the URL.

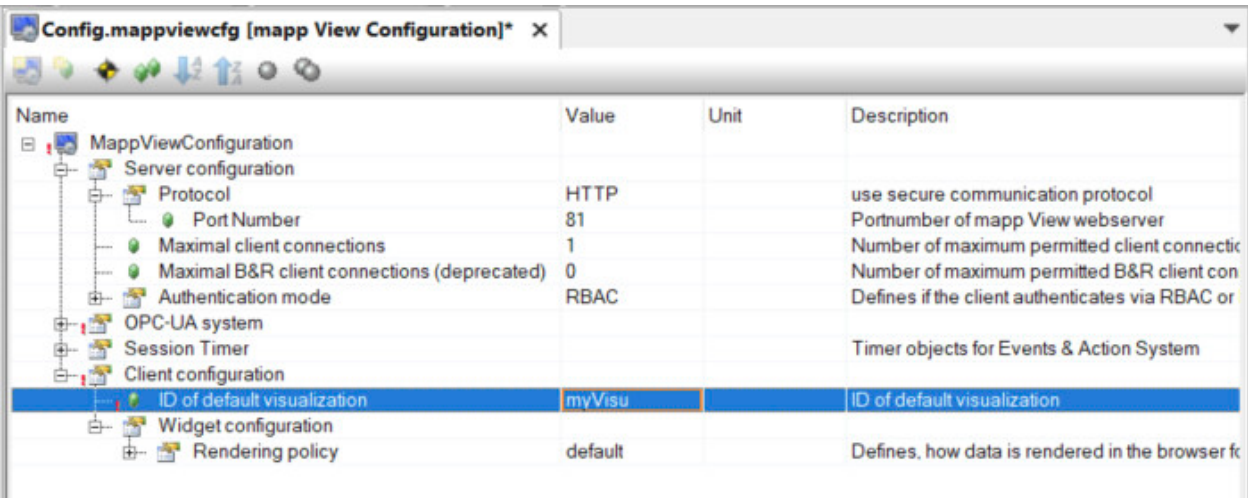


Figure 12: Default HMI Application in mapp View Configuration.

References

None

6.3 Rule A203: mapp configuration file naming

Guidelines

When inserting mapp components from the toolbox, configuration files are added to the Configuration View. The file ending corresponds to the name of the mapp component. Take into account the following considerations when naming the mapp configurations file.

- 1) The file name can freely be chosen.
- 2) The filename can be of maximum 10 characters, due to AR constraints, because each mapp configuration file is transferred as a single binary modules to the AR system.
- 3) Each configuration file should carry a unique name in the project.
- 4) Assign the configurations to the respective packages in the Configuration View.

Use prefixes to indicate the mapp component , followed by the name of the module. For example, AxEjector → Prefix "Ax" for mappAxis config + Name of module.

 This guideline is meant as a suggestion/proposal and not as a mandatory rule.

The prefix can be limited to 4 characters leaving 4 characters for the module name.
Some prefix suggestions for commonly used mapp components are presented in the table below:

mapp component	prefix
Axis	Ax
Axes Group	AxG
Axis Feature	AxF
MpAlarmXCore	Alm
MpAlarmXHistory	AlmH
MpRecipeXml	RcpX
MpRecipeCsv	RcpC

Examples

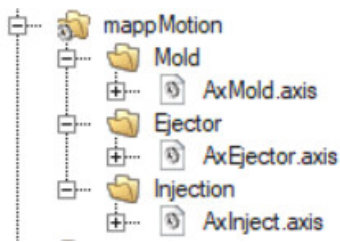


Figure 13: Folder structure in the Configuration View showing several mapp configuration files.

Reasoning

Naming conventions like these increase readability and promote standardization.

Exceptions and comments

Why is there a 10 Character limitation? The 10-Character limit comes from BR-Module-Format, introduced in the 1980s, when memory was in the kilobytes.

References

None

7 Rules A301-A306 Coding principles

Coding principles play a crucial role in software development. Following the guidelines in this section, will help you create robust, and maintainable code. Properly designed code in turn simplifies testing and debugging processes.

7.1 Rule A301: Avoid hard-coded literals

Guidelines

When it is required to parameterize an element, such as the delay time of a timer, it is highly recommended to avoid using hard-coded literals. Instead, parameterize the element with a variable, a member of a structure or a recipe parameter.

Examples

NO:

```
PROGRAM _CYCLIC
    Timer.PT:= T#1s200ms;
END_PROGRAM
```

YES:

```
VAR
    DelayTime : TIME := T#1s200ms;
END_VAR

PROGRAM _CYCLIC
    Timer.PT:= DelayTime;
END_PROGRAM
```

Reasoning

Being able to change parameters and most values during runtime is important. Using a parameter instead of a hard-coded literal provides an efficient way to change the value of this element, without the need to transfer any software changes. Change could be made from the Watch window, via the HMI, by loading a recipe etc.

Exceptions and comments

Sometimes constants are needed for values that are not subject to change during runtime. For example dangerous oxygen levels for humans, value of π (3.1416), etc.

In some cases, it is necessary to check the values from the parameter file against some boundaries or even plausibility. For such cases, you can also use constants.

References

None

7.2 Rule A302: Naming of .var and .typ files

Guidelines

Besides local variables, Automation Studio offers the following scopes for variables and types:

- global variables/types
- package variables/types
- package variables/types with global visibility

The following names should be applied for these files:

- 1) global variables/types
Keep "Global.typ" and "Global.var" which AS assigns as default names when the files are dragged from the toolbox.
- 2) package variables/types
Keep the default names ("Variables.var", "Types.typ"), that AS assigns when the files are dragged from the toolbox.
- 3) package variables/types with global visibility
Name the file according to the usage. Add the tag "(global visibility)" to the description. Typically this option is used to facilitate communication between machine modules (designed as packages in the Logical View).

Examples

1)



Figure 14: Screenshot showing global variables/types

2)



Figure 15: Screenshot showing package variables/types

3)

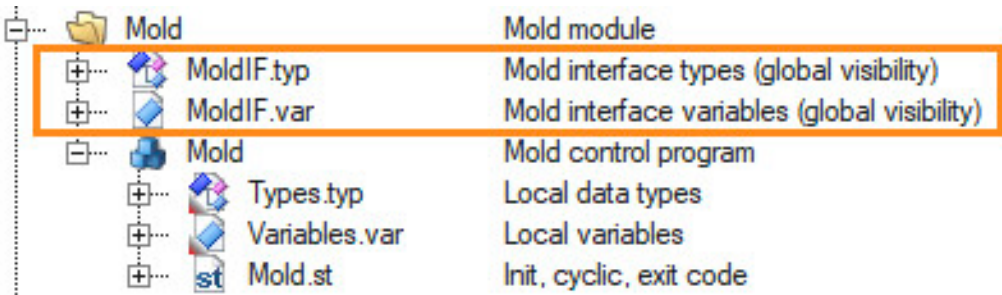


Figure 16: Screenshot showing package variables/types with global visibility

Reasoning

- 1) global variables/types
AS default names generally should be applied wherever feasible.
- 2) package variables/types
AS default names generally should be applied wherever feasible; Package-global visibility is implicitly clear due to position of files; belonging to package (machine unit) also self-explanatory in the Logical View.
- 3) package variables/types with global visibility
The setting is quite hidden in a dialog box and the global visibility should be made more obvious in the Logical View.

Exceptions and comments

In order to make the .var and .typ file in a package visible to other packages, choose the "Global" option for Visibility in the .var or .typ properties window.

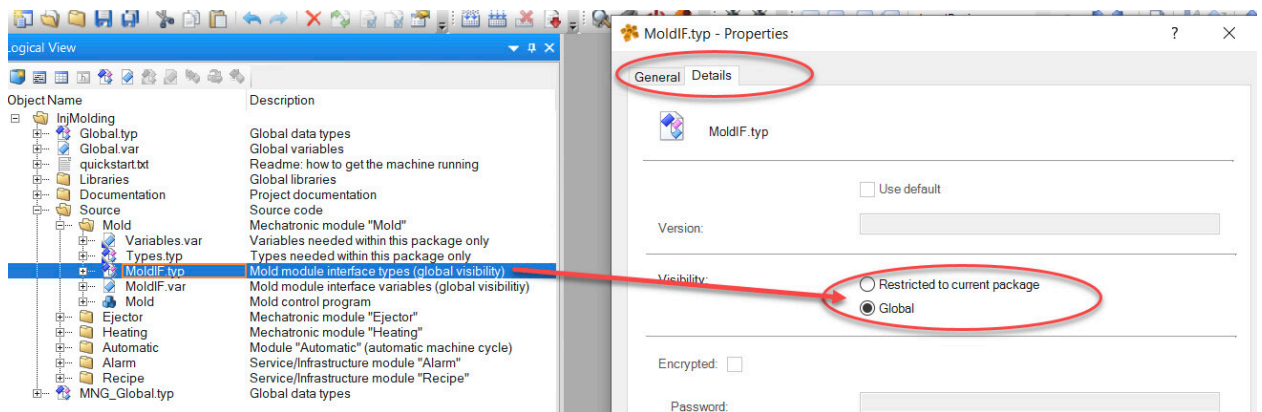


Figure 17: "Global" option for Visibility in the .typ file.

References

None

7.3 Rule A303: Release dynamic resources in the _EXIT

Guidelines

If a program has reserved resources (e.g. dynamically allocated memory) in the _INIT or _CYCLIC program section, it must release these resources again in the _EXIT subroutine.

Examples

An example can be the dynamic memory reservation in the _INIT with the TMP_alloc() function, which should be released in the _EXIT by means of the TMP_free() function.

Most mapp components (especially mapp Services) reserve resources and need to release them in the _EXIT for their correct operation.

```
PROGRAM _EXIT

    // Disable and call mapp FBs

    gAlarmXCoreTank.Enable := FALSE;

    gMpAlarmXCoreTank();

END_PROGRAM
```

Other examples:

- FileOpen → FileClose;
- FileCreate → FileClose;
- DevLink → DevUnlink;

Reasoning

When transferring a program to the PLC, all allocated resources must be released before they are reallocated (note: _EXIT is executed before a program is uninstalled, _INIT and _CYCLIC are executed again after the program is reinstalled on the PLC). If allocated resources or memory are not released properly, they remain allocated, which can lead to problems such as memory overflow over time.

Exceptions and comments

When using mapp Motion or ACP10/ARNCO, it is important to know that the function block is not permitted to be disabled in the EXIT section of the program.

The application of this guideline depends on the Transfer mode: This rule is only applicable if "Execute Init/Exit" (the default setting) is enabled in the transfer settings.

References



B&R Online Help link: [Download behavior](#)

7.4 Rule A304: Don't disable mapp Motion function blocks in the _EXIT

Guidelines

When using mapp Motion or ACP10/ARNC0, it is important to know that the function block is not permitted to be disabled in the EXIT section of the program.

Examples

None

Reasoning

This avoids problems during a download. The cyclic section is stopped with advanced installation setting "Consistent installation". Calling a function block in the EXIT section would not be completed and the download process cannot be completed.

Mandatory for all mapp Motion and ACP10/ARNC0 function blocks.

Exceptions and comments

The different behaviors between mapp Motion and the other mapp Technology Packages, such as mapp Services or mapp Control, must be taken into account.

References



B&R Online Help link: [Download behavior](#)

7.5 Rule A305: Avoid bottom-up dependencies

Guidelines

Lower-level modules should not depend on higher-level modules.

Examples

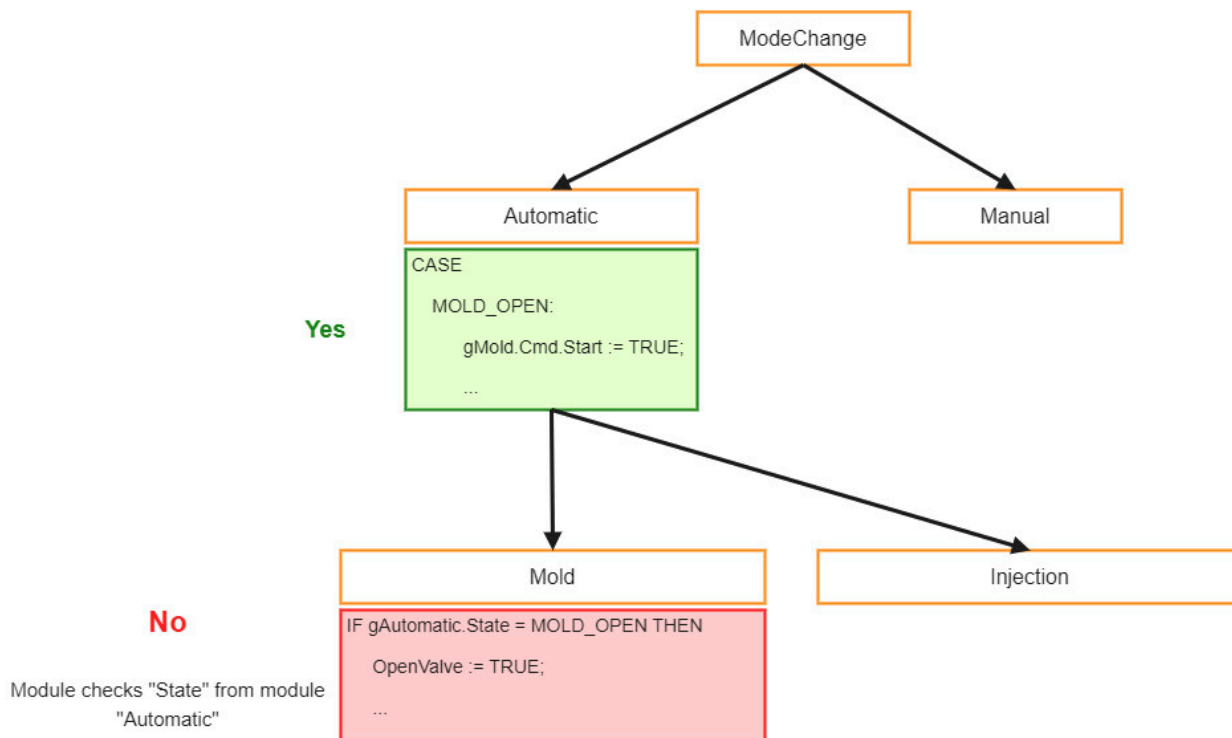


Figure 18: Diagram showing dependencies between modules.

Reasoning

Avoiding bottom-up dependencies facilitates reusability of code in other applications.

Machine applications should follow a modular approach (see Rules A401-A4xx: Modularity). The modules will typically have some hierarchical order.

For example, the automatic sequence decides when to start or stop the single machine components in a dedicated order. The automatic sequence therefore resides in the software architecture on a higher level than the mechatronic machine components.

Higher-level modules should control lower-level modules. Lower-level modules should not include any knowledge about higher-level modules.

Exceptions and comments

To be able to achieve top-down dependencies and avoid bottom-up dependencies, all modules must provide well-defined interfaces, see rule [Rule A401: Well-defined interfaces](#)

References

[Rule A401: Well-defined interfaces](#)

7.6 Rule A306: Set parameters before giving command

Guidelines

Assign values to the parameters needed by a certain command before setting that command to TRUE and not after setting the command.

Examples

This section is for relevant code examples.

NO:

```
gConveyor.Cmd.Move := TRUE;  
gConveyor.Par.Velocity := gManual.Par.ConveyorVelocity;  
gConveyor.Par.Acceleration := gManual.Par.ConveyorAcceleration;  
gConveyor.Par.Deceleration := gManual.Par.ConveyorDeceleration;
```

YES:

```
gConveyor.Par.Velocity := gManual.Par.ConveyorVelocity;  
gConveyor.Par.Acceleration := gManual.Par.ConveyorAcceleration;  
gConveyor.Par.Deceleration := gManual.Par.ConveyorDeceleration;  
gConveyor.Cmd.Move := TRUE;
```

Reasoning

Provides better readability of code.

Exceptions and comments

None

References

None

8 Rules A401-A408: Modularity

What is modularity in software?

Software modularity is decomposing software into smaller pieces with standardized interfaces. It is analogous to modularity for hardware. We want to create products by combining reusable chunks of code, so you only implement a feature or functionality once and then **maximize reuse**.

Software modularity in machine automation

We have all heard of object orientation, reusability and extendibility in correlation to modularity. But there is more to it than meets the eye. Having a modular structure in your software projects means much more.

A modular software makes it possible to develop parts of a machine in parallel and not in a sequence.

Automation is going further away from the old concept of one engineer one machine. Nowadays, there are multiple teams (mechanics, electricians, software developers) which work together with each other. Single modules of an assembly line can be fully assembled and developed in parallel to the main controller. Each station can be exchanged by a station of a similar type without adjusting anything other than that specific module.

A modular software that follows the machine design enables us to identify and solve issues faster.

If the software design follows the mechanical and electrical design, an application engineer can find the issue immediately simply by navigating and isolating a module. If for example, a reaction does not occur due to a broken sensor, all the application engineer has to do is to navigate to the module, check the IO outputs/inputs and then go to the Physical View and tell the other engineer the name of the module in their technical/electrical drawings.

A modular software makes it possible to work together without interfering with each other.

A big issue that projects often have is working together in source-control using Automation Studio. Two people edit an OpcUa file and merge it, but the changes of one are lost. These kinds of issues occur on a daily basis. A good modular and structured Automation Studio project makes sure that such conflicts do not happen.

A modular software makes it easy to get new engineers up to speed.

By having a clear structure to follow, a new engineer joining the project finds it easier to identify their work packages and find issues/bugs in already implemented modules. A modular software design doesn't only make it easier to work inside Automation Studio, but affects the engineering workflow as a whole like planning work packages, implementing features and tests, ordering replacement hardware or giving project status updates to the management.

The following guidelines should help an Application Engineer in structuring their Automation Studio project in a way that makes it as easy and efficient as possible to work together as a team using source control, project planning tools and with other teams (mechanical, electrical and even ordering).

8.1 Rule A401: Well-defined interfaces

Guidelines

It is recommended, that the modules (or call it "units", "components", which could be the single tasks or a group of tasks in a package) in your application, should provide a dedicated and well-defined communication interface to the rest of the application (following the encapsulation principle of OOP).

When 2 modules communicate with each other, they should exchange as little information as possible. Communication should be limited to the absolute minimum.

The machine module ("unit", "component", ...) should be operated only through its interface (e.g. in Watch window during program development, or by other modules in an hierarchic order, e.g. automatic sequence tasks starts sequentially certain machine components). The implementation details should be kept internal.

It is recommended to divide the interface structure into Commands (Cmd), Parameters (Par) and Status information. Commands and Parameters are set by upper-level (controlling) modules and determine what the lower-level (controlled) module should execute.

Status information is flowing the other way round from the lower-level (controlled) module to the upper-level (controlling) module.

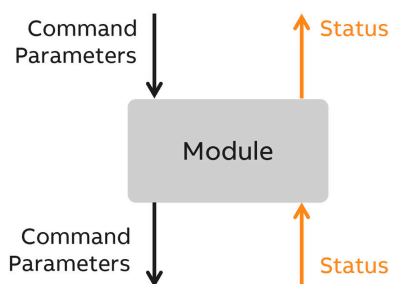
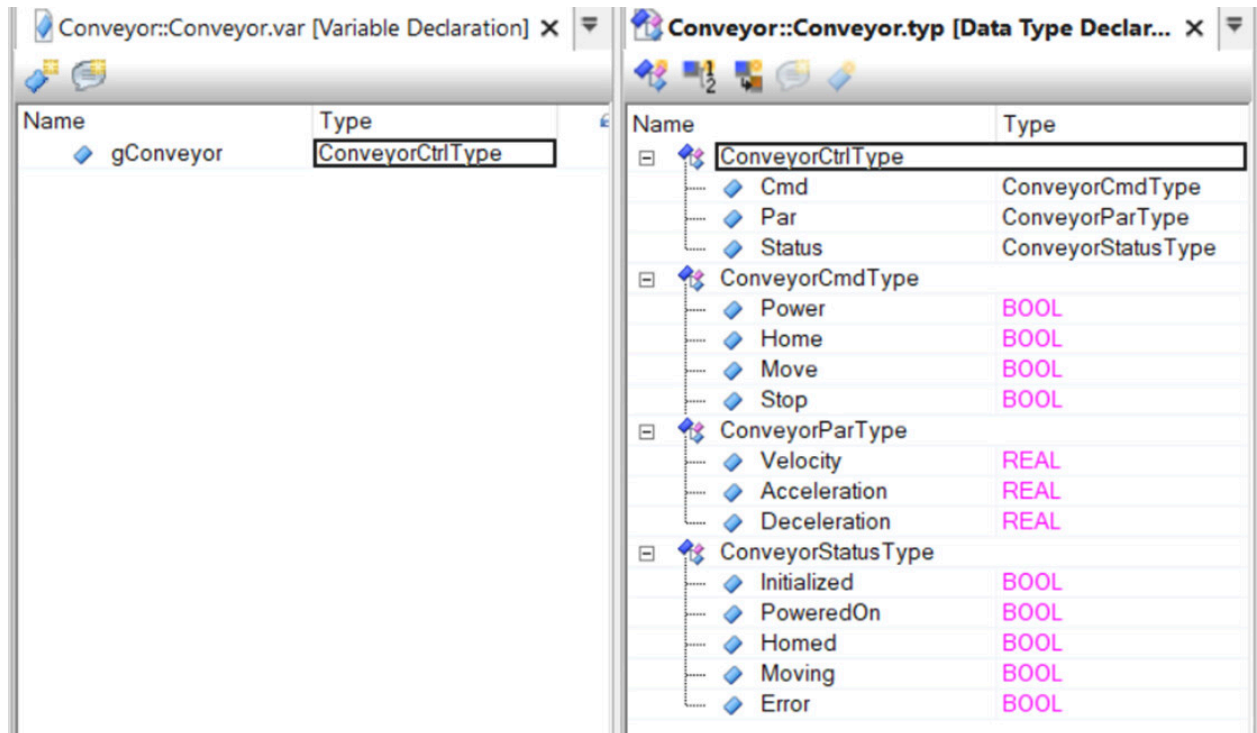


Figure 19: Interfaces of a module.

Examples

A conveyor belt can be powered on, homed, moved and stopped. First, the conveyor belt is programmed and tested in Watch window. Then, an automatic sequence is implemented that operates the conveyor belt (together with other machine modules). Both manual tests (Watch) and triggering movements from the automatic cycle should be performed via the dedicated interface structure.



Reasoning

Well-defined interfaces provide the following benefits:

- encourages modular design of your application
- increases quality because of a clear distinction between internal implementation details and the representation to the outside
- limits inter-task dependencies to a minimum
- increases testability

Exceptions and comments

The design of inter-task communication highly depends on local demands, best practices and customer specifications. Therefore the guidelines should be adopted as far as possible but can be adapted to the specific project constraints. A separate interface structure for the HMI should be avoided. The HMI should also only access the same (one and only) interface structure. The HMI should not access any internal variables.

To get a better overview of which interface variables are being used in the HMI, follow [Rule A408: Unique *.binding and *.eventbinding file for per *.content file \(mapp View\)](#)

Consider creating a local copy of the interface structure in the module, follow [Rule A408: Unique *.binding and *.eventbinding file for per *.content file \(mapp View\)](#)

References

[Rule A408: Unique *.binding and *.eventbinding file for per *.content file \(mapp View\)](#)

8.2 Rule A402: Enable input for each software module

Guidelines

In terms of modularity, the application software should be organized in self-sufficient and reusable modules. It should be possible to activate/deactivate a module via a variable in the module interface.



This guideline is meant as a suggestion/proposal and not as a mandatory rule.

Examples

The `_CYCLIC` routine of the module could contain at the beginning some logic like:

```
PROGRAM _CYCLIC

IF NOT gModule.Cmd.Enable AND NOT gModule.Status.BeingDisabled
    RETURN; // Abort program execution
```

Reasoning

Following are the benefits of being able to activate or deactivate a module during runtime:

- This procedure greatly simplifies the activation/deactivation of programs during commissioning for testing purposes.
- It makes it very easy to activate/deactivate a module depending on the machine configuration.

Exceptions and comments

None

References

None

8.3 Rule A403: Create a local copy of the interface structure

Guidelines

Create a local copy of the interface structure in the module to prevent issues related to task class timings and interruptions.



This guideline is meant as a suggestion/proposal and not as a mandatory rule.

Examples

Option 1

- Read/write the global structure in the cyclic sub program within task.

```
PROGRAM _CYCLIC
```

```
    Mold.Par := gMold.Par;  
    Mold.Cmd := gMold.Cmd;  
  
    // Cyclic code  
    // ...  
  
    gMold.Status := Mold.Status;
```

```
END_PROGRAM
```

Option 2

- Read/write to the global structure using the variable mapping editor (PV mapping).

Reasoning

Creating a local copy of the interface structure provides following benefits:

- prevents issues related to task class timings and interruptions
- state/status of a task cannot be externally overwritten or manipulated
- timing behavior of the application does not change when task execution order is modified.

Exceptions and comments

Cyclic assignment or copying of huge data structures could affect PLC performance in a negative way.

References

None

8.4 Rule A404: IO-module names in Physical View should correspond to design documents

Guidelines

In electrical wiring diagrams, typically specific names are given to the IO modules of the PLC. Name the IO modules in the Physical View of Automation Studio identical to the names assigned in the electrical wiring diagram.

Examples

Depending on how the customer decides to name their IO Modules in the circuit diagram:

- Instead of AI4222a → HV105 (High Voltage cabinet; rack 1; position 5)
- Instead of DI4372c → LV312 (Low Voltage cabinet; rack 3; position 12)

Reasoning

The start of a software project usually follows the design of a circuit diagram. To validate that IOs are correctly wired and sensors are working the automation engineer checks those together with an electrical engineer and circuit diagrams.

It is quite common in an Automation Studio that people name the IOs in the Physical View as AI4222, AI4222a, AI4222b, etc.. This creates confusion during checking and working together with other teams.

To avoid this confusion, it is recommended to name the physical IO modules identical to the ones in the circuit diagram.

Exceptions and comments

None

References

None

8.5 Rule A405: Mirror Logical View folder structure in Configuration View

Guidelines

The Logical View folder structure should be applied in a similar way to the Configuration View.

The folder structure of Logical View should be applied for every mapp Package (mappControl, mappMotion, mappServices, ...) and on a higher level for organizing *.sw, *.iom and *.pvm files in the Configuration View.

Examples

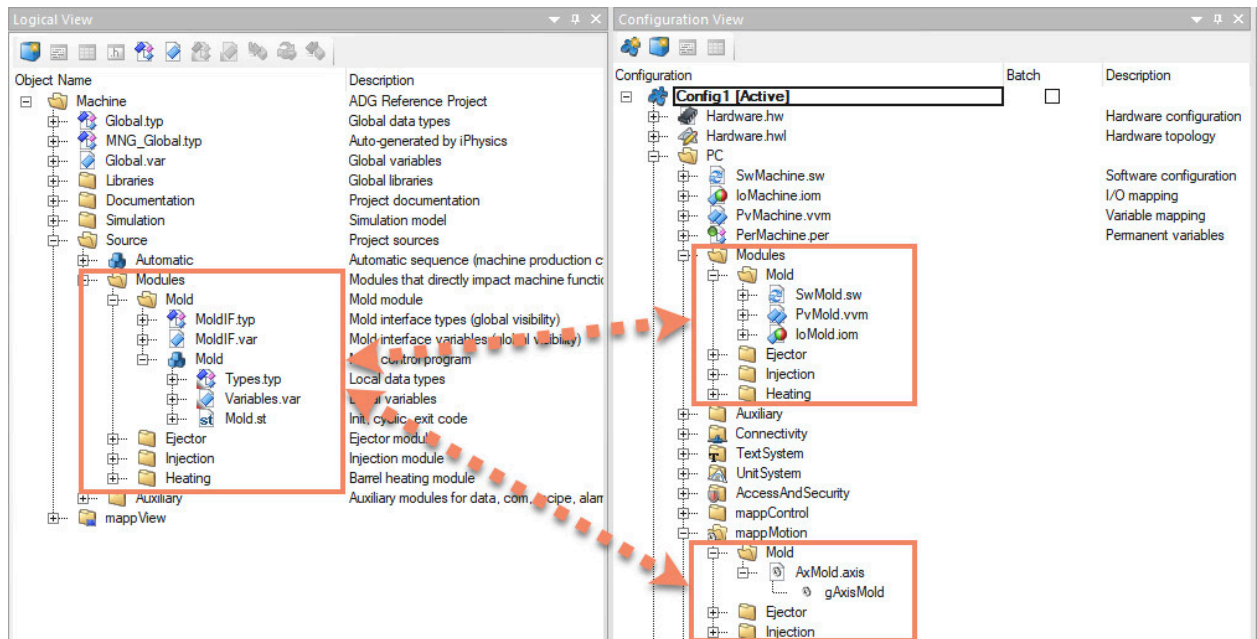


Figure 20: The folder structures in the Logical View with the corresponding folder structures in the Configuration View.

Reasoning

Consistent structuring helps find files and features faster and simplifies collaboration in version control systems by reducing the potential for merge conflicts.

Exceptions and comments

This rule is currently not applicable for OpcUaMap (.uad) files, because the AS compiler issues a warning if an OpcUaMap (.uad) file is outside the default folder. Therefore, the separated .uad files should all be placed in the default folder in Configuration View (Connectivity/OpcUA), see also rule [Rule A407: Unique OpcUaMap \(.uad\) file for every module](#)

References

[Rule A407: Unique OpcUaMap \(.uad\) file for every module](#)

8.6 Rule A406: Unique *.sw, *.iom, *.vwm file per module

Guidelines

In terms of modularity, the application software should be organized in self-sufficient and reusable modules. Each module should have its own *.sw, *.iom, *.vwm within the top-level module folder that got introduced by the "Rule A405: Mirror Logical View folder structure in Configuration View"

Examples

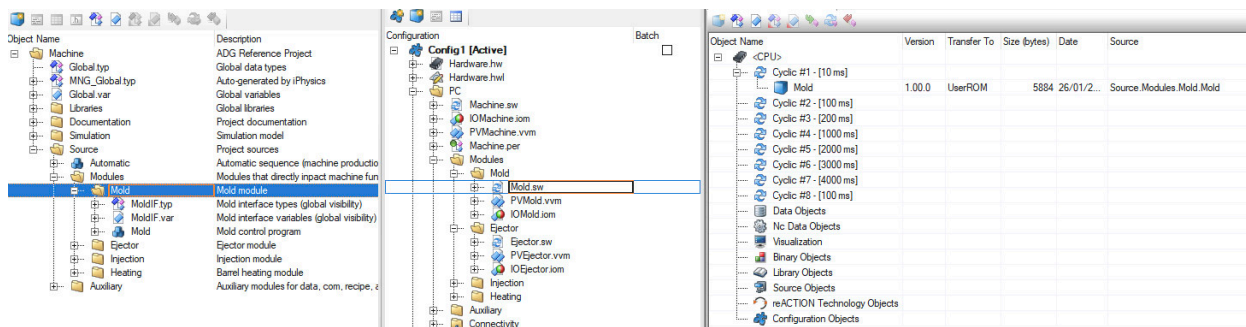


Figure 21: The package in Logical View with its own .sw file in the Configuration View.

Reasoning

Structuring the project into multiple markup and mapping xml-based files will simplify collaboration in version control systems by reducing the potential for merge conflicts.

By clicking on the CPU in Physical View and then Software, you can still see all the deployed tasks in one window if needed.

Exceptions and comments

*.iom files have to be maintained manually, because the AS editor adds the mappings always into the top most file! Use a text editor and then copy & paste the mappings into the desired file.

*.iom and *.vwm have to have a prefix (eg. IO, PV), because they are compiled and put onto the CPU target the same way as a task (.br files must have a unique name) and they can't share the same name due to that.

References

[Error number: 9222](#)

8.7 Rule A407: Unique OpcUaMap (.uad) file for every module

Guidelines

Every module should have its own separate and unique OpcUaMap (.uad) file where only programs that belong to that module get their OpcUA nodes enabled.

Examples

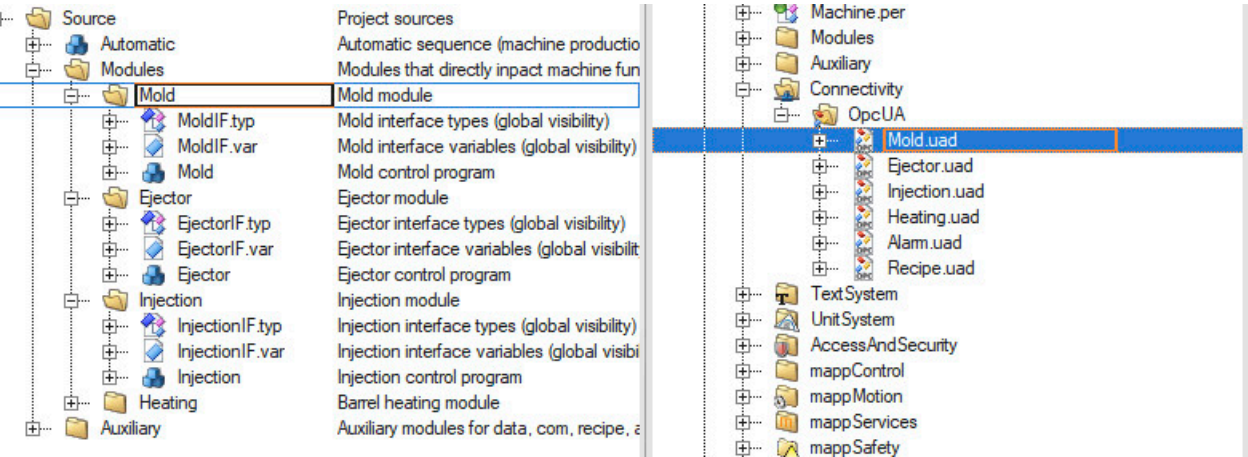


Figure 22: The module in Logical View with its unique OpcUaMap (.uad).

Reasoning

Merging of XML files is basically impossible and separating modules into specific Sub-OpcUa files makes merge conflicts less common. This simplifies collaboration in version control.

Exceptions and comments

The "Rule A405: Mirror Logical View folder structure in Configuration View" is currently not applicable for OpcUaMap (.uad) files, because the AS compiler issues a warning if an OpcUaMap (.uad) file is outside the default folder. Therefore, the separated .uad files should all be placed in the default folder in Configuration View (Connectivity/OpcUA).

References

None

8.8 Rule A408: Unique *.binding and *.eventbinding file for per *.content file (mapp View)

Guidelines

Every mappView Content should have its separate and unique .binding and .eventbinding file in Configuration View. Mirror the folder structure from Logical View to the Configuration View:

- Pages in Logical View should be matched by Packages in Configuration View
- The names of Contents inside the pages in Logical View should match the names of .binding and .eventbinding files

Examples

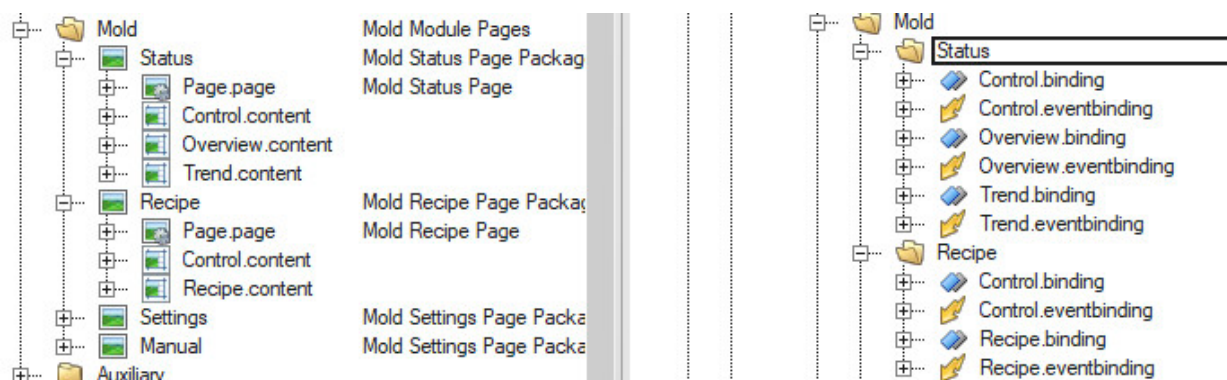


Figure 23: Each mappView Content with its unique .binding and .eventbinding files in the Configuration View.

Reasoning

- Separate files make it easier to find and manage bindings.
- This approach simplifies collaboration in version control systems by reducing the potential for merge conflicts.

Exceptions and comments

Some contents that are just using direct bindings for values do not need Eventbinding files. If the page does not need an Eventbinding file make sure to delete it.

References

Rule A405: Mirror Logical View folder structure in Configuration View

9 Rules A501-A502: Documentation

Comprehensive documentation empowers your team, streamlines communication, and ensures project resilience. The guidelines presented in this section help in providing clarity, maintainability, and effective communication.

9.1 Rule A501: Include a "Readme" file

Guidelines

Provide a "Readme.txt" file for every project. This file could contain following information:

- Quickstart guide
- Prerequisites for testing the project (recipe file locations, directories for ARsim, etc.)
- Instructions for testing
- ...

This file should be placed in the top most folder where the *.apj file is located.

Examples

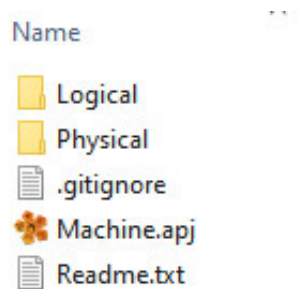


Figure 24: The "Readme.txt" file in the project structure.

Reasoning

Files with the name "Readme.txt" will be auto-resolved by any Git remote repo manager.

feature/application-design

coding-and-design-guidelines / ADGSample /

Source	Description
..	
Logical	
Physical	
.gitignore	Initial Commit
Machine.apj	Update project based on proposed modularity guidelines for logical and configuration view
README.txt	Update project based on proposed modularity guidelines for logical and configuration view
README.txt	
AUTOMATIC MODE 1. Open iPhysics Simulation (IMM.iphz in "Simulation" package) 2. Open Watch of Automatic program 3. Automatic.Cmd.Run = TRUE RECIPE 1. c:/temp folder needed (recipes are saved in this folder) 2. Open Watch of Recipe program 3. gRecipe.Cmd.Save = TRUE (edge sensitive) 4. "IMM.xml" is created ALARM 1. c:/temp folder needed (alarm history is exported to this folder) 2. Open Watch of Alarm program 3. gAlarm.Cmd.ExportHistory = TRUE (edge sensitive) 4. "Alarms_XXX.csv" is created HEATING 1. Open Watch of Heating program 2. gHeating.Cmd.PowerOn = TRUE	



You can also use markdown format Readme.md to be able to format the content.

Exceptions and comments

None

References

None

9.2 Rule A502: List abbreviations used in the project

Guidelines

List all abbreviations used in the project. This document should be available under the "Documentation" folder of the project.

Examples

List the abbreviations in a .tmx file and name the file "Abbreviations.tmx". Benefits of using this file format:

- can be directly opened in Automation Studio
- additional languages can be added
- new entries can be made easily in tabular format
- unique Text IDs ensure unique abbreviations

Reasoning

Listing abbreviations avoids confusion for those who work on the project later.

Exceptions and comments

None

References

None

10 Rules A601-A601: Testing

label

Testing is a critical aspect of ensuring software quality. Here are some general practices to follow:

- Set specific objectives for testing. These objectives guide decisions throughout the testing phase.
- Identify potential risks related to functionality, security, performance, and usability.
- Create comprehensive test plans: Specify test scope, test cases, and expected outcomes.
- Document testing environments and configurations.
- Automate repetitive and time-consuming test scenarios.
- Continuously test existing functionality after code changes. Regression testing prevents unintended side effects.
- Ensure compliance with security standards.

10.1 Rule A601: Use the IEC Check library during development

Guidelines

Use the IEC Check library during development to check for

- range violations for arrays
- division by zero
- ...

The effect of the library is to put the CPU into service mode with an exact report of the error and its cause, containing for example the responsible program name.



This guideline is meant as a suggestion/proposal and not as a mandatory rule.

Examples

None

Reasoning

None

Exceptions and comments

The IEC Check library functions should not be used in machines where a restart or service mode is not allowed.

Generally speaking, the IEC Check library functions should be used during development and testing, but not when a machine is in (critical) production.

The option `-D_IGNORE_CHECKLIB` can be enabled in the additional build options, so that the functions in this library are not used.

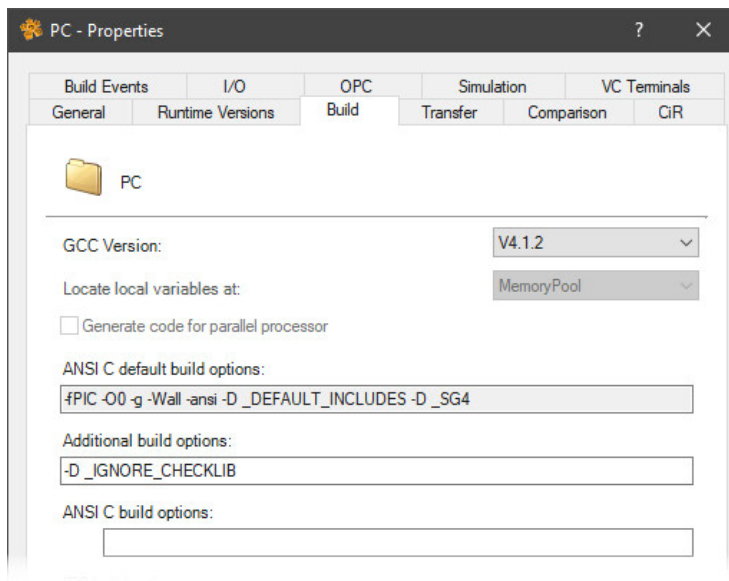


Figure 25: The option `-D _IGNORE_CHECKLIB` is enables in the additional build options, to prevent using IEC Check library functions.

References

[Programming / Libraries / IEC Check library](#)

11 Rules A701-A702: Usability

11.1 Rule A701: Make SDM available on the HMI

Guidelines

Make the SDM (System Diagnostics Manager) available to the user on the HMI to be able to access the B&R inbuilt diagnostics pages in a web browser.



This guideline is meant as a suggestion/proposal and not as a mandatory rule.

Examples

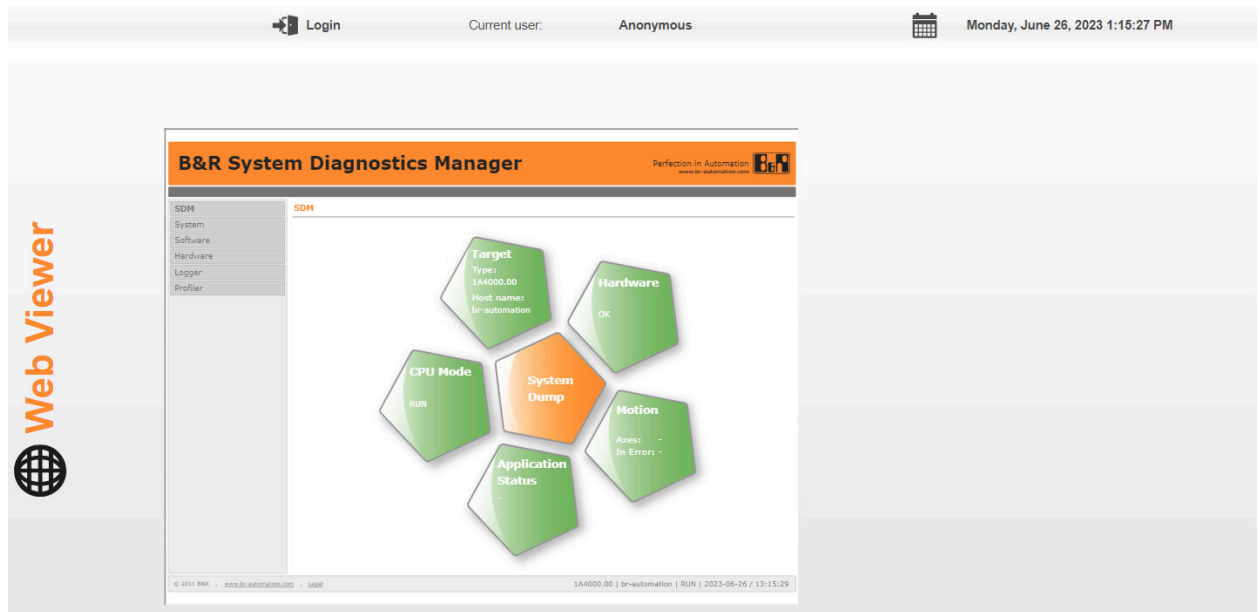


Figure 26: SDM as seen on the HMI.

Reasoning

Providing the SDM on the HMI offers preliminary troubleshooting possibilities and system information to the machine operator.

The SDM pages are inbuilt and it is possible to access them directly on a web browser without much engineering effort.

Exceptions and comments

Add and configure WebXs for diagnostics and configuration of mapp components in the SDM.

References

None

11.2 Rule A702: Global variables availability in Watch window

Guidelines

To make a variable with a global scope available in the Watch window of a task it has to be called at least once inside the corresponding task.

Examples

Global variable not called in task

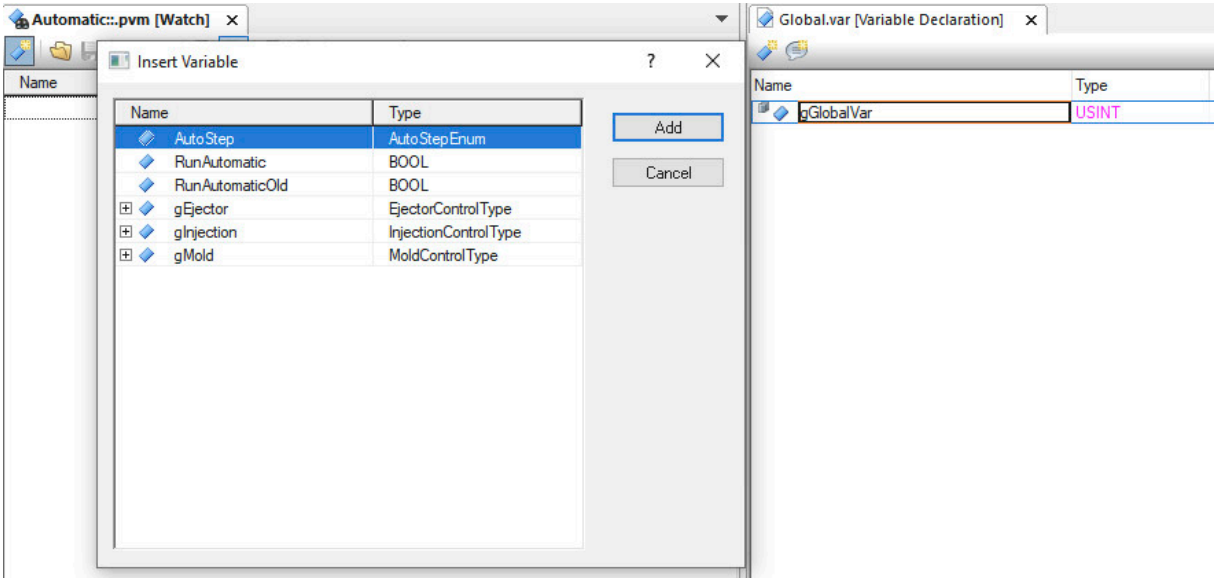


Figure 27: Watch window when global variables are not called in the task.

Global variable called in task

A global variable can be called in the INIT routine to make it visible in the Watch window for that task.

```
PROGRAM _INIT
    gGlobalVar;
END_PROGRAM
```

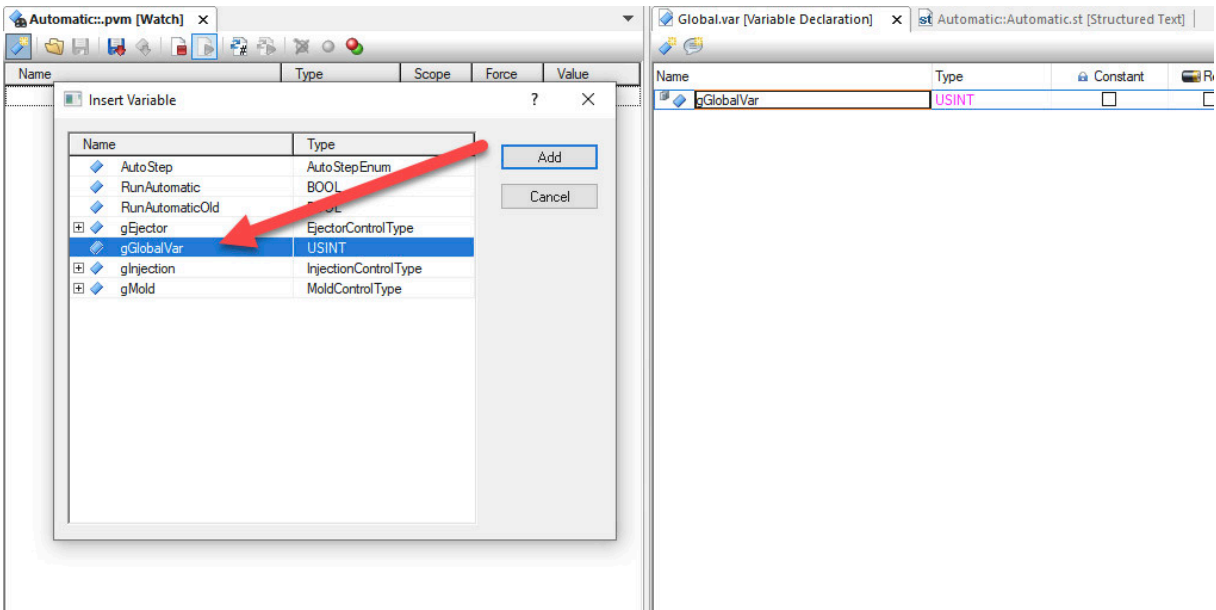


Figure 28: Watch window when global variables are called in the task.

Reasoning

Some global variables need to often be monitored in the Watch window of a module, even though they are actually never called inside that module.

Exceptions and comments

This goes hand in hand with [Rule A403: Create a local copy of the interface structure](#)

Doing a PV-Mapping from a local to a global structure often means that that global structure is not being called inside the task it is actually being used. Although in some cases, it would be necessary to monitor this global structure in the task's Watch window.

References

None

Automation Academy

Your knowledge advantage

The Automation Academy provides **targeted training** courses for our customers as well as for our own employees. Expand your skills in the field of automation technology and learn to independently implement **efficient automation solutions** using B&R systems.

Decide for yourself which **learning concept** you prefer!



Classroom learning



Virtual classroom learning



Online courses

Classroom learning

B&R offers **standard seminars** at all B&R locations. Services include seminar documents, effective communication of learning content by experienced trainers and an Automation Diploma. A combination of group work and self-study provides the high level of flexibility needed to maximize the learning experience.

Virtual classroom learning

Remote Lectures supplement B&R's continuing education portfolio with a virtual classroom, offering an alternative to our on-site seminars. Selected content from our standard seminars is offered online. In addition to remote learning methods, powerful simulation tools and secure remote maintenance are used.

Online courses

Take control of the content and learn at your own pace. With B&R **online courses**, you can take your first steps in the world of B&R automation at any time. Based on a comprehensive narrative, you will independently work out how to use our products. The mix of different media allows a logical sequence to be followed when learning as well as a selective choice of information to be used as a reference tool.

Contact

Would you like additional training? Are you interested in finding out what the B&R Automation Academy has to offer? If so, this is the right place.

Access additional information here:

<https://www.br-automation.com/de/academy/>

Enjoy your next training course!



AutomationAcademy





B&R
Industrial Automation GmbH
A member of the ABB Group
B&R Straße 1
5142 Eggelsberg, Austria
office@br-automation.com

t +43 7748 6586-0
f +43 7748 6586-26

br-automation.com

